



Figure 1: Common threats

research and tools are often absent. Third party vendors typically do not port their formalised assessment, monitoring, auditing and masking tools because the demand does not justify the costs.

Tasks such as collecting user permissions, manual comparison of settings in relation to established policies and keeping up to date with patches are very time-consuming and error-prone, and can be automated by open source software developers. Access control and use of SSL are features often provided by vendors, but advanced security such as thwarting SQL injection and memory exploits falls under the developer's purview. Smaller firms usually have trouble justifying the cost of developing these features.

Comparing a few open source DBs for security

The security vulnerabilities of four open source DBs are compared in the areas of data encryption, client-server communication path encryption and authentication, inter-cluster authentication and encryption, script injection and DDoS attacks.

a) **MongoDB:** All data is unencrypted and stored as plain text. The MongoDB ports — 27017 and 28017— have no data encryption. In shared mode, MongoDB does not support authentication, which can be activated

in standalone mode, though. Attackers can use JavaScript to hack MongoDB.

- b) **Cassandra:** Data files are stored without encryption. The current version of Cassandra uses SSL. However, it is not enabled by default and hence the data in motion would be unencrypted. SQL injections are easily accomplished using the Cassandra Query Language. DDoS attacks can also be easily made since Cassandra runs on one thread per client, and attackers can easily make fake connections and bog down resources.
- c) **CouchDB:** CouchDB does not have automatic data encryption. Client-to-server communications have built-in SSL support. The DB has JSON

injection vulnerabilities as it uses JSON to manipulate data. CouchDB is also known to be affected by DDoS attacks.

- d) **Redis:** Redis does not support encryption for data at rest or in motion. Redis is used for caching and hence is designed to function rapidly. Therefore, it does not have many components for its security. It has been reported, though, that Redis is not prone to injection or DDoS attacks.

Security can be enhanced for the above (and other) NoSQL open source databases by encrypting data at the application or OS levels. The use of reverse-proxy would help set up SSL connections for encryption. Handling SQL injections and DDoS attacks requires more effort from developers.

With the shortage of third party tools, the open source software industry is taking on the onus of developing security features for its products. According to the CTO of Ingres, the company has built its own security features with a clearly written guide for distribution. Similarly, Postgres has declared that it has tightened its security focus in the latest releases of the database by including support for RADIUS server authentication and LDAP support. In the coming years, it looks like the OSS industry will focus more on the security for its products and invite support from third party tools. **END** 🐧

References

- [1] JAQM, Issue 4, Vol. 4, Database Security Attacks and Control Methods – Emil Burtescu
- [2] IJERT Special Issue – 2017, Database Security: Attacks, Threats and Access Control Methods
- [3] A comparison of the level of security of the Top 5 open source NoSQL databases – Preecha Noiumkar, Tawatchai Chomsiri
- [4] Wikipedia

By: Anita Nair

The author works at EYGDS, India. Her areas of interest include cloud computing, cybersecurity, software design patterns, DevOps, computer networking and infrastructure, emerging technologies and research. She is a senior member of IEEE and a life member of CSI. She maintains a technical blog at <https://mindovertechnologyblog.wordpress.com/>.